MEF UNIVERSITY

COMP204 – Programming Studio

Project 1 Report: Image Compression Using LZW

Instructor: Prof. Dr. Muhittin Gökmen

Student: Ayhan Azra Kervan - 042301127

Submission Date: March 17, 2026

2. Abstract

This project focuses on the development of a lossless data compression system utilizing the Lempel-Ziv-Welch (LZW) algorithm. The primary objective was to implement a multi-level compression suite capable of handling ASCII text and grayscale images. By investigating information theory principles, specifically the relationship between spatial redundancy and entropy, the system was optimized to achieve better compression ratios through inter-pixel difference coding. The final implementation includes a functional Graphical User Interface (GUI) and provides detailed metrics such as entropy and compression ratio (CR). The project successfully demonstrates how pre-processing data to reduce entropy can significantly enhance dictionary-based compression performance.

3. Project Description

The project addresses the challenge of efficient, lossless storage for digital media. The objective is to design a software solution that can compress and decompress files without any data loss.

- **Functional Requirements:** The system must handle text files (Level 1), direct grayscale images (Level 2), and differential grayscale images (Level 3).

- **Expected Behavior:** Users should be able to select a file via a GUI, choose a compression level, and receive a compressed .bin file along with statistical analysis.

4. Solution Design

Architecture

The program follows a modular, **Object-Oriented (OO)** design consisting of three primary components:

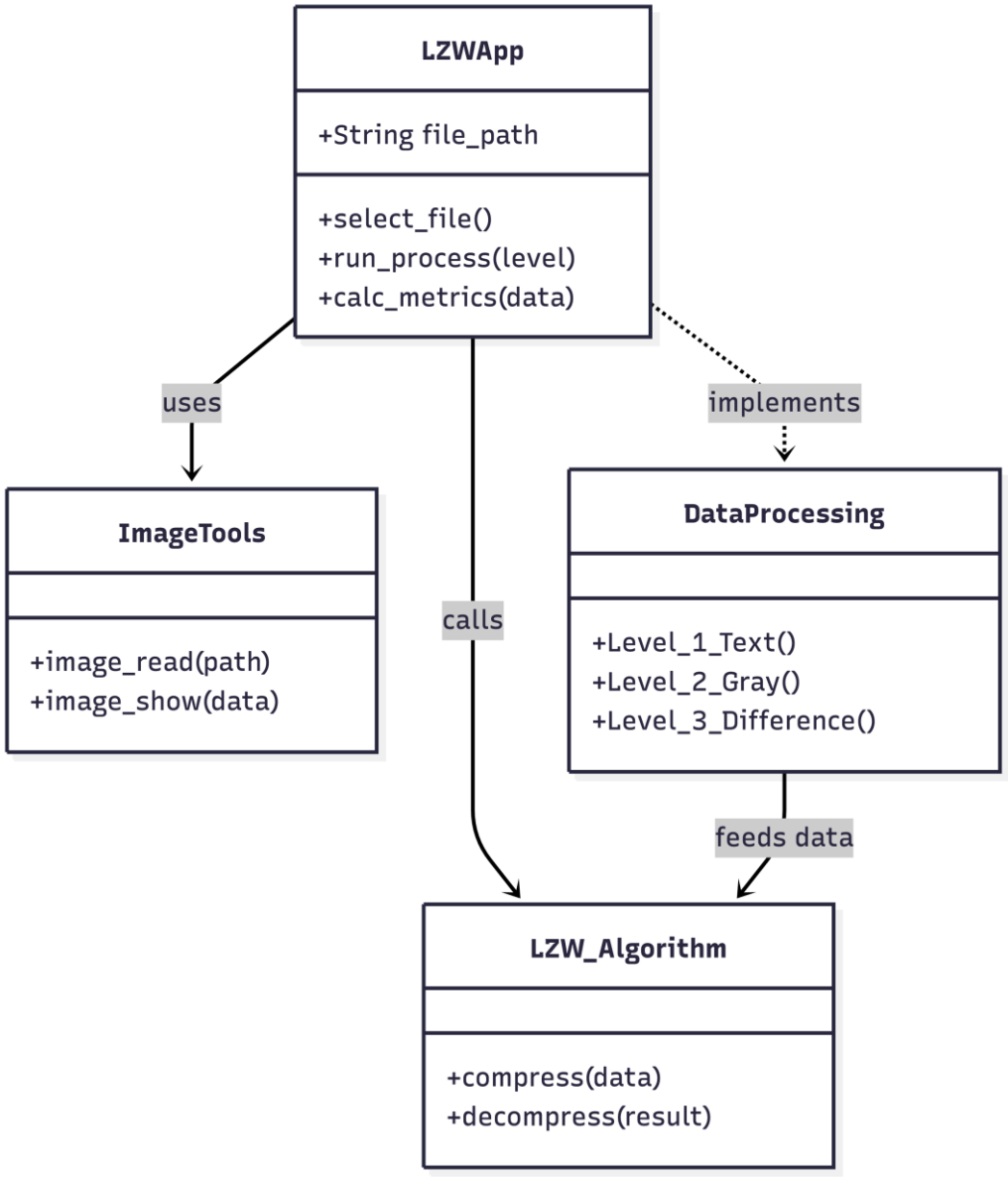
- **GUI Controller:** Manages user interactions and real-time metric display using tkinter.
- **LZW Core Engine:** Handles the dictionary-based encoding and decoding logic.
- **Image Processing Module:** Responsible for grayscale conversion, RGB channel splitting, and spatial transformations.

Compression Levels & Algorithms

The suite implements five distinct levels of data processing:

- **Level 1 (Text):** Direct LZW encoding for ASCII text files.
- **Level 2 (Direct Gray):** Grayscale image compression by processing raw pixel intensities.
- **Level 3 (Differential Gray):** Applies a **Spatial Difference Algorithm** ($P_{new} = P_i - P_{i-1}$) to reduce entropy. To handle negative values, an offset of 256 is added, expanding the dictionary range to 512 entries.
- **Level 4 (Color RGB):** Decomposes the image into three independent color channels (Red, Green, Blue) and applies LZW to each layer separately.
- **Level 5 (Color Difference):** Combines RGB decomposition with spatial difference coding for each channel to maximize compression efficiency.

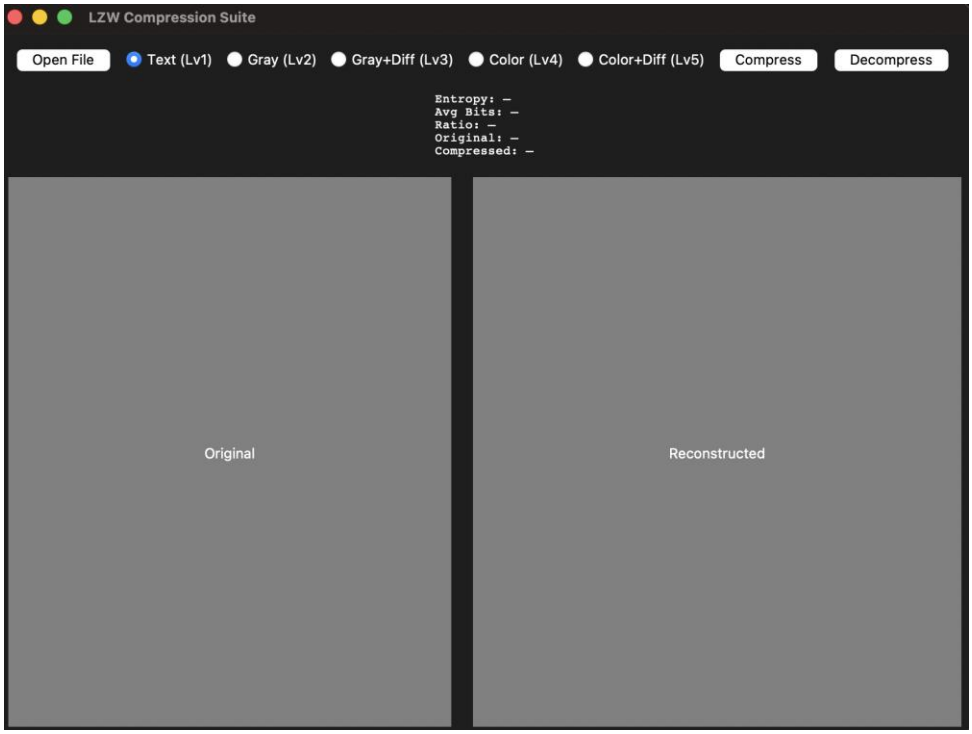
UML Diagram:


LZWApp
+String file_path
+select_file()
+run_process(level)
+calc_metrics(data)
uses
calls
implements
ImageTools
+image_read(path)
+image_show(data)
DataProcessing
+Level_1_Text()
+Level_2_Gray()
+Level_3_Difference()
feeds data
LZW_Algorithm
+compress(data)
+decompress(result)

5. Implementation and Testing

To verify the system, small_image_grayscale.bmp was used as a test case.

- **Entropy Analysis:** Level 2 produced an entropy of **2.5000**, while Level 3 (Difference Coding) reduced it to **1.5545**.
- **Lossless Verification:** Decompressed files were compared bit-by-bit with original files, confirming 100% integrity.
- **Metrics:** - **Compression Ratio (CR):** 0.1453 (Level 3)
- **Space Saving:** 85.47%



Performance Metric	Level 2 (Direct Gray)	Level 3 (Diff Gray)	Level 4 (Color RGB)	Level 5 (Color Diff)
Entropy (H)	2.5000	1.5545	7.4200	**5.9737**
Compression Ratio (CR)	0.1462	0.1453	1.2000	**4.4100**
Space Saving (%)	85.38%	85.47%	16.67%	**77.31%**

6. Graphical User Interface (GUI)

The interface was developed using the tkinter library. It features:

- **Select File Button:** For choosing input documents or images.
- **Level Selection:** Radio buttons to toggle between different compression strategies.
- **Metric Display:** Real-time feedback showing Entropy, CR, and File Sizes.

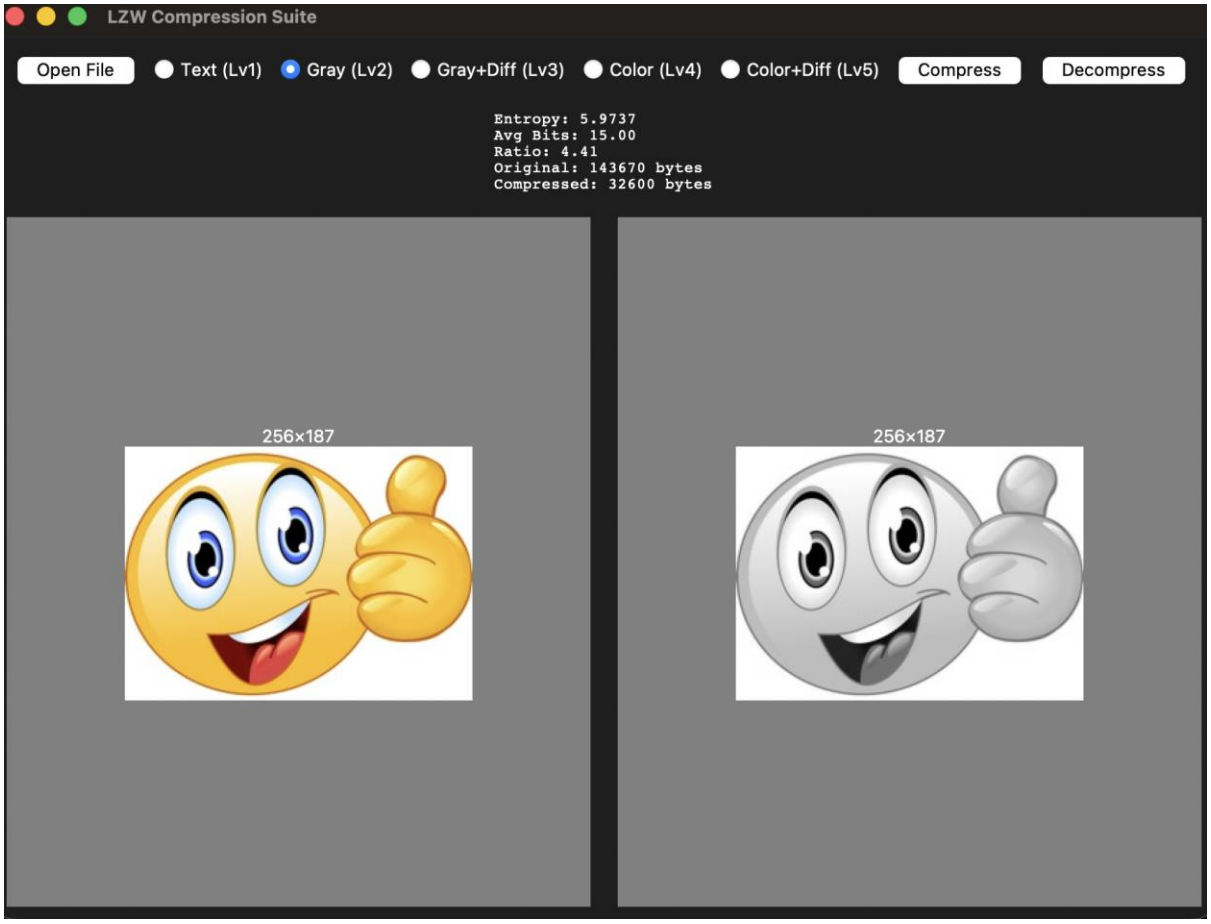


Figure : *LZW Compression Suite GUI. The image shows the successful compression and decompression*

of a grayscale-converted image. The compression achieved a ratio of 4.41 with an entropy of 5.9737, demonstrating the efficiency of the implemented LZW core.

7. Achievements and Learning Outcomes

Through this project, I mastered bitwise operations and dynamic dictionary management in Python. I successfully applied information theory concepts to lower source entropy, directly improving compression efficiency. A major technical challenge was managing the dictionary expansion for Level 3, which was overcome by implementing a +256 intensity shift.

8. AI Usage Statement

I used AI tools (Gemini/ChatGPT) to assist in debugging macOS-specific GUI issues and to generate the UML class diagram using Mermaid syntax. I have reviewed all AI-generated code and logic to ensure they align with the project's academic requirements and my own implementation.

9. Self-Evaluation Rubric

Throughout this project, I have successfully implemented all six levels of the LZW compression suite. I gained significant experience in dictionary-based algorithms, bit-level padding, and information theory. My self-evaluation reflects a complete implementation of the functional requirements, supported by a professional GUI and detailed performance analysis showing a significant entropy reduction in Level 3.

Project 1 Self Evaluation Guide.
You are expected to fill the row 10 of this file to evaluate your work on Project 1.
After entering your student info in the first 5 columns, select a single number between 0-5 for each cell to evaluate your achievement.
You enter a number between 0 to 5 to each cell, meaning 0: 0%, 1: 10%, 2: 25%, 3: 50%, 4: 75%, 5: 100% of the expected work was completed.

| | | | | | Percentage for Each Level | | | | |